

ITSIMPLERA INSTITUTE
Internship Program — Blue Team Track

WEEK 1 — TASK BRIEF

Task Type: Setup + Exploration • Difficulty: Beginner

TASK SUBMISSION

Week 1

Wazuh SIEM — Manager, Windows Agent & Linux Agent Deployment

Manager Installation • Multi-Endpoint Agent Registration • File Integrity Monitoring • Live
Security Event Verification

Intern Name	Muzaker Khan
Role	Blue Team Intern
Task	Week 1 — Wazuh SIEM Deployment
Task Released	June 27, 2026
Submission Deadline	July 4, 2026
Date Performed	July 1, 2026
Manager (Server & Dashboard)	Kali Linux 2025.4 — 192.168.179.128
Windows Agent (End-point)	Windows 10 VM — 192.168.179.142
Linux Agent (End-point)	Kali Linux VM — 192.168.179.143

Contents

1. Introduction & Objectives	2
2. Network Verification	4
3. Wazuh Manager Installation on Kali Linux	7
4. Windows 10 Agent Deployment	9
5. Linux (Kali) Agent Deployment	13
6. Troubleshooting: Agent/Manager Version Mismatch	18
7. File Integrity Monitoring (FIM) on the Linux Agent	20
8. Live Security Events — Windows Agent	22
9. Reflection & Conclusion	24

1. Introduction & Objectives

Security Information and Event Management (SIEM) is a foundational technology in modern cybersecurity operations. A SIEM platform collects, normalises, and correlates log data from across an organisation's infrastructure — endpoints, servers, network devices, and applications — in real time. By centralising event data, the platform enables security analysts to detect anomalous behaviour, investigate incidents, and produce compliance-ready audit trails from a single dashboard.

Wazuh is a fully open-source, enterprise-grade SIEM and Extended Detection and Response (XDR) platform. It follows a manager-agent architecture: the Wazuh Manager acts as the central collection and analysis engine, while lightweight Wazuh Agents run on monitored endpoints and forward events to the manager over an encrypted, authenticated channel. The manager also ships a web-based dashboard (built on OpenSearch Dashboards) that visualises alerts, agent health, and compliance posture.

This lab extends the standard single-agent deployment into a small, heterogeneous monitoring environment: one Kali Linux host runs the full Wazuh stack (indexer, manager, Filebeat, dashboard), a Windows 10 virtual machine is enrolled as a Windows endpoint agent, and a second, independent Kali Linux virtual machine is enrolled as a Linux endpoint agent. File Integrity Monitoring (FIM) is then configured and validated on the Linux agent, and live security events are verified from both endpoints through the Wazuh dashboard.

1.1 Learning Objectives

- Understand the Wazuh manager-agent architecture and the role each component plays in a SIEM deployment.
- Verify a VMware NAT network so that a Windows endpoint, a Linux endpoint, and a Kali manager can all communicate on the same subnet.
- Install the complete Wazuh stack (indexer, manager, Filebeat, dashboard) on Kali Linux using the official all-in-one install script.
- Deploy the Wazuh agent on a Windows 10 endpoint (MSI) and on a separate Kali Linux endpoint (APT), pointing both at the manager IP.
- Register each agent on the manager using the `manage_agents` CLI tool, generating a unique cryptographic authentication key per agent.
- Diagnose and resolve an agent/manager version-mismatch failure by upgrading the manager stack.
- Configure and validate File Integrity Monitoring (Syscheck) on the Linux agent, and observe file-added / file-modified / file-deleted alerts in the dashboard.
- Generate representative security events on the Windows agent and verify them in near real time on the Wazuh web dashboard.

1.2 Lab Environment

Component	Details
Wazuh Manager Host	Kali Linux 2025.4 x64 (kali-linux-2025.4-vmware-amd64)
Manager IP Address	192.168.179.128
Manager Resources	4 GB RAM, 4 vCPU, 80.1 GB disk — VMware NAT
Wazuh Version (Manager)	Installed as 4.7.5 (all-in-one), later upgraded in-place to 4.14.6
Windows Agent Host	Windows 10 VM (“Windows_10_ITSimplera”)
Windows Agent IP	192.168.179.142
Windows Agent Resources	6 GB RAM, 2 vCPU, 60 GB NVMe disk — VMware NAT
Wazuh Agent Version (Windows)	4.14.5 (Windows MSI)
Windows Agent Name / ID	win10 / ID 001
Linux Agent Host	Kali Linux VM (independent instance from the manager)
Linux Agent IP	192.168.179.143
Wazuh Agent Version (Linux)	4.14.6 (APT package)
Linux Agent Name / ID	klin / ID 002
Network Type	VMware NAT — all three VMs share the same 192.168.179.0/24 subnet
Dashboard URL	https://192.168.179.128 (port 443)
Virtualisation	VMware Workstation

2. Network Verification

2.1 Confirming All VMs Share a Common Subnet

Before any software is installed, all three virtual machines must sit on the same network segment so they can reach one another. In VMware Workstation, the hardware settings of the Windows 10 VM are opened first.

Objective: Confirm all three VMs are on the same VMware NAT network so they can reach each other.

Method: Check VMware hardware settings for each VM — Network Adapter must show NAT for all three.

Required: All VMs must be on the 192.168.179.0/24 subnet before any Wazuh installation begins.

As shown in Figure 1, the Windows 10 VM (labelled `Windows_10_ITSimplera`) is configured with 6.0 GB of memory, 2 processors, a 60 GB NVMe hard disk, and a Network Adapter set to NAT. This places the VM on the same VMware-managed subnet that the Kali manager occupies.

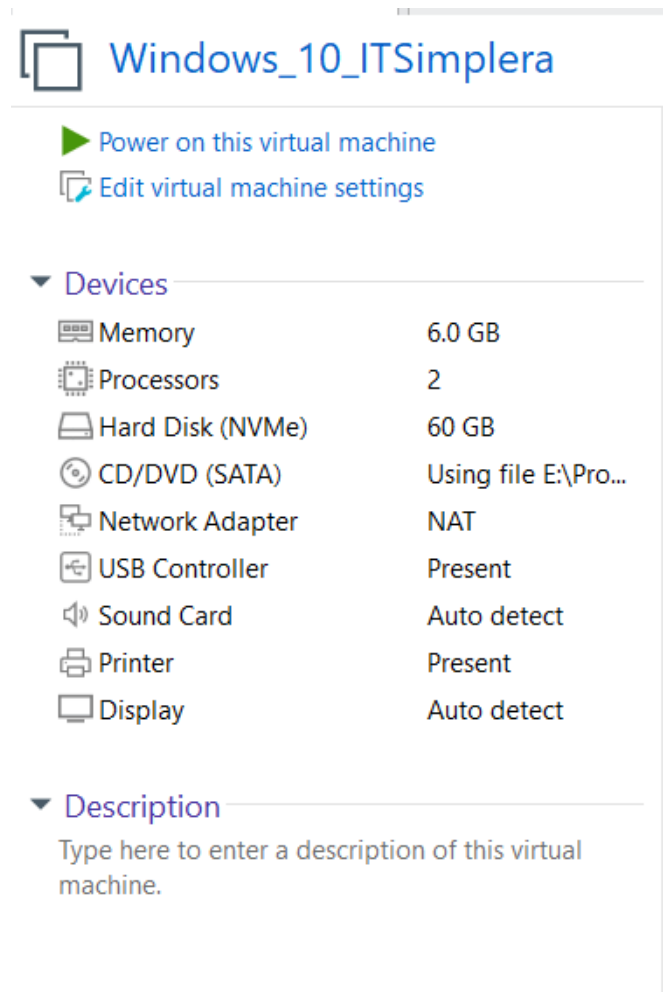


Figure 1: VMware Workstation hardware settings for the Windows 10 VM (Windows_10_ITSimplera). The Network Adapter is set to NAT, with 6.0 GB RAM, 2 processors, and a 60 GB NVMe disk.

The same verification is performed for the Kali Linux manager VM. Figure 2 shows the hardware settings for `kali-linux-2025.4-vmware-amd64`: 4 GB RAM, 4 processors, an 80.1 GB SCSI hard disk, and the Network Adapter also set to NAT — placing it on the identical subnet as the Windows endpoint.

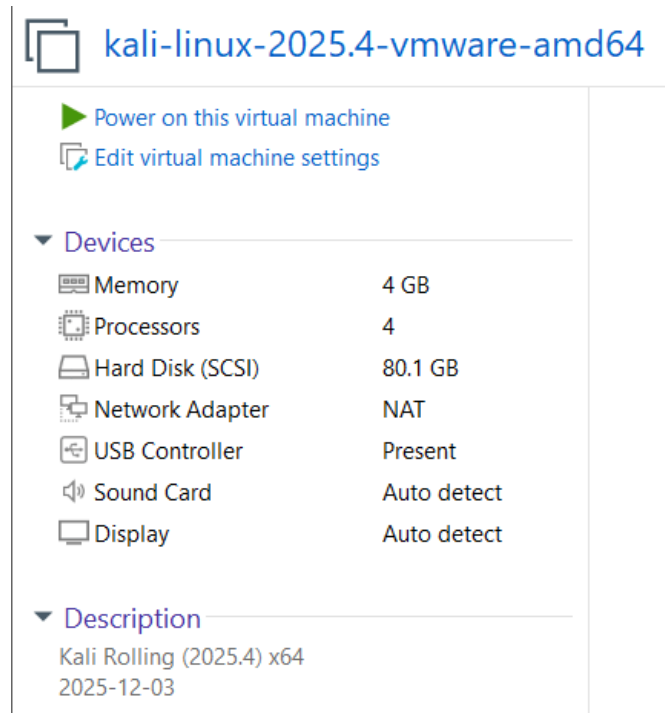


Figure 2: VMware hardware settings for the Kali Linux 2025.4 manager VM. Network Adapter is NAT, RAM is 4 GB, disk is 80.1 GB (Kali Rolling 2025.4 x64).

The second Kali Linux VM used as the Linux agent (`klin`) is provisioned identically on the same NAT network and later confirmed reachable by `ping` in Section 6.1 (Figure 12).

Parameter	Windows 10 VM	Kali Manager VM	Kali Linux Agent VM
Network Mode	NAT	NAT	NAT
IP Address	192.168.179.142	192.168.179.128	192.168.179.143
Role in Lab	Wazuh Agent (Windows endpoint)	Wazuh Manager + Dashboard	Wazuh Agent (Linux endpoint)

3. Wazuh Manager Installation on Kali Linux

Objective: Install the complete Wazuh stack on the manager Kali VM using the official automated install script.

Commands:

```
curl -s0 https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash wazuh-install.sh -a --ignore-check
```

Flag -a: all-in-one — installs indexer, manager, Filebeat, and dashboard in one run.

Flag --ignore-check: bypasses hardware pre-flight checks — needed on a VM-sized Kali box.

Result: Wazuh 4.7.5 installed; dashboard reachable at <https://192.168.179.128>.

On the manager's terminal, the official Wazuh install script for the 4.7 branch is downloaded with `curl`, then executed with elevated privileges. The `-a` flag triggers the all-in-one installation mode, which installs the Wazuh indexer (OpenSearch-based), the Wazuh manager daemon, Filebeat, and the Wazuh dashboard web UI in a single automated run. The `--ignore-check` flag is required because the installer's hardware pre-flight checks are calibrated for production server hardware and would otherwise block installation on a VM.

As shown in Figure 3, installation begins with the assistant confirming Wazuh version 4.7.5, redirecting verbose logging to `/var/log/wazuh-install.log`, adding the Wazuh APT repository, installing dependencies, generating configuration files, creating the `wazuh-install-files.tar` cluster key archive, and starting the Wazuh indexer installation. The WARNING about hardware checks being ignored is expected given the `--ignore-check` flag.

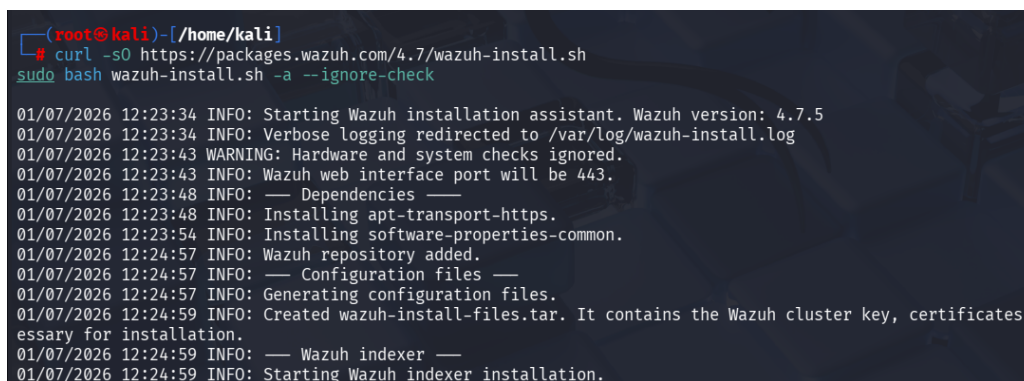
A terminal window showing the execution of the Wazuh installation script. The prompt is (root@kali)-[/home/kali]. The commands entered are curl -s0 https://packages.wazuh.com/4.7/wazuh-install.sh and sudo bash wazuh-install.sh -a --ignore-check. The output shows a series of INFO and WARNING messages. The WARNING message states: 'WARNING: Hardware and system checks ignored.' The INFO messages show the progress of the installation: 'Starting Wazuh installation assistant. Wazuh version: 4.7.5', 'Verbose logging redirected to /var/log/wazuh-install.log', 'Wazuh web interface port will be 443.', 'Dependencies', 'Installing apt-transport-https.', 'Installing software-properties-common.', 'Wazuh repository added.', 'Configuration files', 'Generating configuration files.', 'Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, necessary for installation.', 'Wazuh indexer', and 'Starting Wazuh indexer installation.'

Figure 3: Manager terminal at the start of the Wazuh 4.7.5 all-in-one installation: repository added, configuration files generated, cluster key archive created, and indexer installation beginning.


```

01/07/2026 12:53:34 INFO: Wazuh dashboard installation finished.
01/07/2026 12:53:34 INFO: Wazuh dashboard post-install configuration finished.
01/07/2026 12:53:34 INFO: Starting service wazuh-dashboard.
01/07/2026 12:53:35 INFO: wazuh-dashboard service started.
01/07/2026 12:54:32 INFO: Initializing Wazuh dashboard web application.
01/07/2026 12:54:32 INFO: Wazuh dashboard web application not yet initialized. Waiting...
01/07/2026 12:54:50 INFO: Wazuh dashboard web application not yet initialized. Waiting...
01/07/2026 12:55:05 INFO: Wazuh dashboard web application initialized.
01/07/2026 12:55:05 INFO: — Summary —
01/07/2026 12:55:05 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
User: [REDACTED]
Password: [REDACTED]
01/07/2026 12:55:05 INFO: Installation finished.

(root@kali)-[/home/kali]
#

```

Figure 4: Installation summary confirming the dashboard is accessible on port 443, with the User and Password fields redacted for security. “Installation finished.” confirms the full stack is running.

Component	Status
Wazuh Indexer (OpenSearch)	Installed and running
Wazuh Manager	Installed and running
Filebeat	Installed and running
Wazuh Dashboard	Installed and running, web app initialised
Dashboard URL	https://192.168.179.128 (port 443)
Installed Version	Wazuh 4.7.5 (later upgraded — see Chapter 7)

4. Windows 10 Agent Deployment

Objective: Install the Wazuh Windows agent on the Windows 10 VM and configure it to point at the Kali manager.

Agent Package: `wazuh-agent-4.14.5-1.msi` (downloaded from docs.wazuh.com)

Manager IP in `ossec.conf`: 192.168.179.128 **Port:** 1514 **Protocol:** TCP

Config file path: `C:\Program Files (x86)\ossec-agent\ossec.conf`

4.1 Downloading the Windows Agent

On the Windows 10 VM, the Wazuh documentation page for deploying agents on Windows endpoints is opened. Figure 5 shows the download in progress in the browser's download bar: `wazuh-agent-4.14.5-1.msi` (5.7 MB), status *Done*. The page confirms the agent supports systems from Windows XP through the latest releases, and that administrator privileges are required for installation.

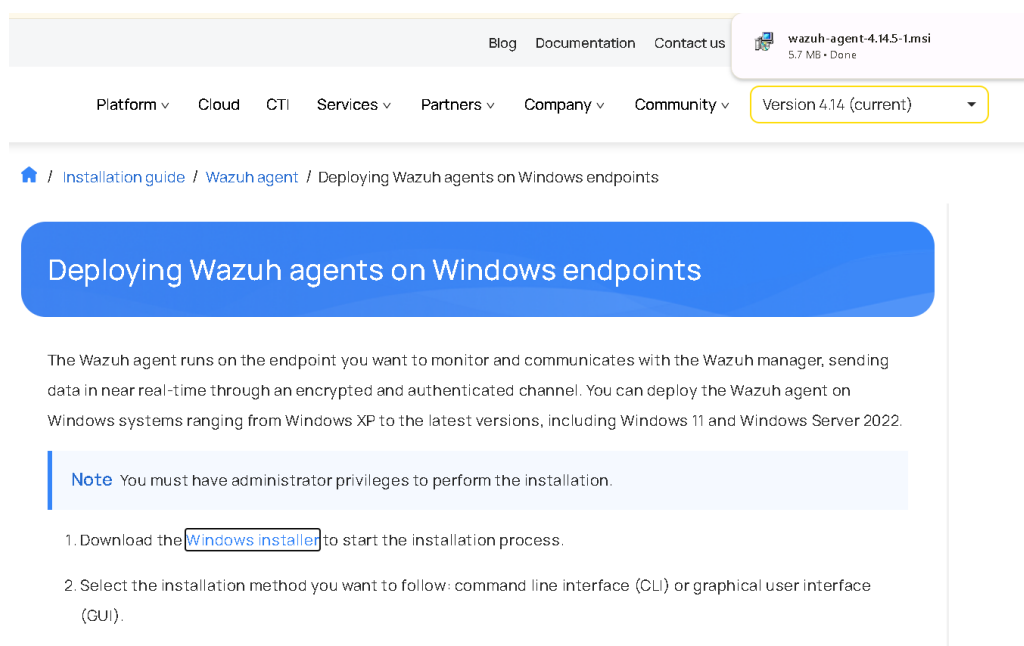


Figure 5: Wazuh documentation page “Deploying Wazuh agents on Windows endpoints” with `wazuh-agent-4.14.5-1.msi` (5.7 MB) fully downloaded in the browser toolbar.

4.2 Running the Agent Setup Wizard

The downloaded MSI is launched, opening the Wazuh Agent Setup graphical installer. Figure 6 shows the installer mid-way through its “Installing Wazuh Agent” step, confirming the agent files are being placed under `C:\Program Files (x86)\ossec-agent\`.

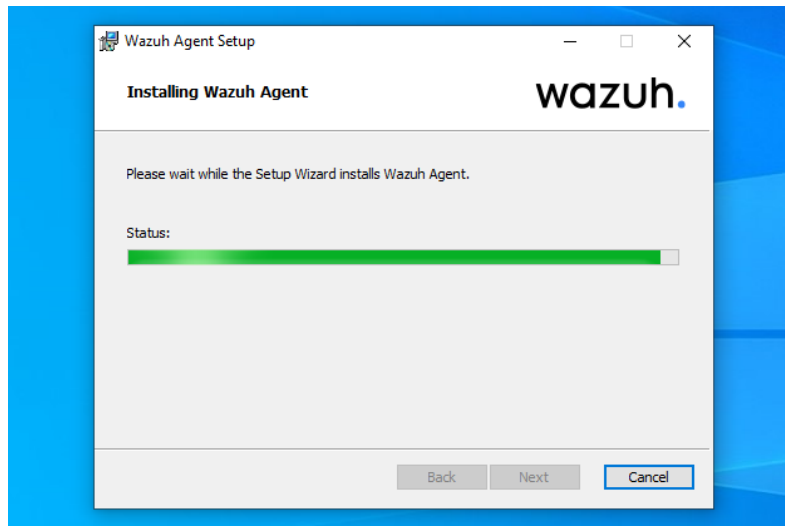


Figure 6: Wazuh Agent Setup wizard running on the Windows 10 VM, showing the installation progress bar.

4.3 Configuring the Manager Address

After installation, `ossec.conf` is opened in Notepad. Inside the `<client><server>` block, the `<address>` tag is set to `192.168.179.128`, the port to `1514`, and the protocol to `TCP`. The `<config-profile>` is set to `windows, windows10`, ensuring the correct OS-specific ruleset is applied, and `<crypto_method>` is `aes`, encrypting the agent-manager channel.

```
<ossec_config>
  <client>
    <server>
      <address>192.168.179.128</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>windows, windows10</config-profile>
    <crypto_method>aes</crypto_method>
    <notify_time>20</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
  </client>
```

Figure 7: `ossec.conf` open in Notepad on the Windows 10 agent: manager address `192.168.179.128`, port `1514`, protocol `TCP`, config-profile `windows, windows10`, crypto_method `AES`.

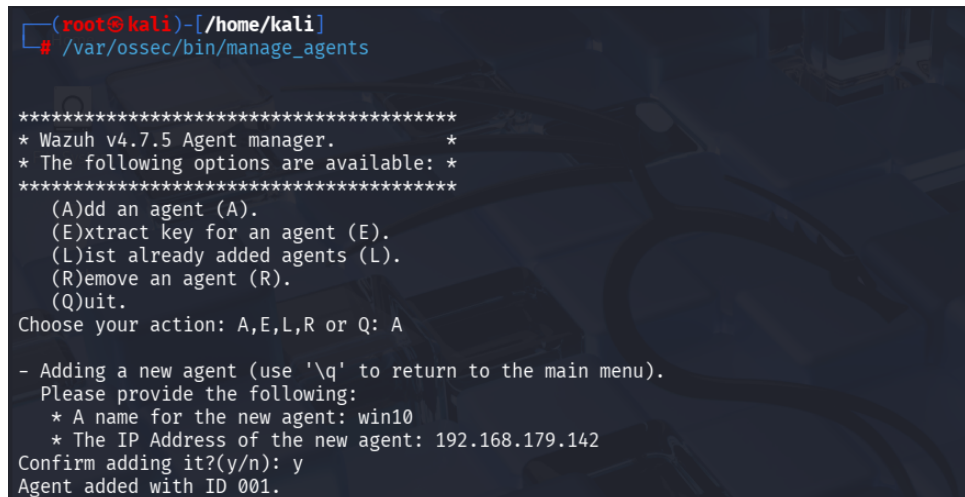
Config Parameter	Value	Purpose
<code><address></code>	<code>192.168.179.128</code>	IP of the Wazuh Manager (Kali Linux)
<code><port></code>	<code>1514</code>	Default Wazuh agent communication port
<code><protocol></code>	<code>tcp</code>	Reliable transport for log forwarding
<code><config-profile></code>	<code>windows, windows10</code>	Applies Windows-10-specific detection rules
<code><crypto_method></code>	<code>aes</code>	AES encryption on the agent-manager channel
<code><auto_restart></code>	<code>yes</code>	Agent restarts automatically on config changes

4.4 Registering the Windows Agent on the Manager

On the manager, the registration tool is launched as root:

```
/var/ossec/bin/manage_agents
```

Option A (Add an agent) is selected. The agent is named `win10` with IP `192.168.179.142`. After confirming with `y`, the tool reports **Agent added with ID 001**.



```
(root@kali)-[/home/kali]
# /var/ossec/bin/manage_agents

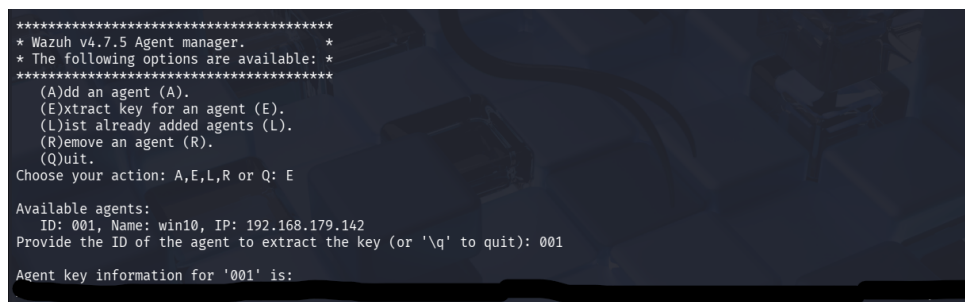
*****
* Wazuh v4.7.5 Agent manager.          *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: A

- Adding a new agent (use '\q' to return to the main menu).
  Please provide the following:
  * A name for the new agent: win10
  * The IP Address of the new agent: 192.168.179.142
Confirm adding it?(y/n): y
Agent added with ID 001.
```

Figure 8: `manage_agents` on the Kali manager: option A selected, agent named `win10` with IP `192.168.179.142`, added with ID 001.

4.5 Extracting the Authentication Key

The same tool is reopened and option E (Extract key) selected. With agent ID 001 entered, the manager displays the full base64-encoded authentication key, which is copied for import on the Windows agent. The key is redacted in Figure 9 to protect the live cryptographic material.



```
*****
* Wazuh v4.7.5 Agent manager.          *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: E

Available agents:
  ID: 001, Name: win10, IP: 192.168.179.142
Provide the ID of the agent to extract the key (or '\q' to quit): 001

Agent key information for '001' is:
[REDACTED]
```

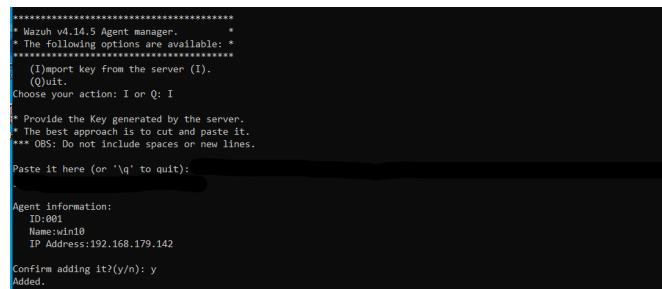
Figure 9: `manage_agents` option E: agent 001 (`win10`, `192.168.179.142`) key extracted (redacted for security).

4.6 Importing the Key on Windows

On the Windows 10 VM, an Administrator Command Prompt is opened and the working directory changed to the agent installation folder:

```
cd "C:\Program Files (x86)\ossec-agent"
manage_agents.exe
```

The Windows build of `manage_agents` (v4.14.5) offers option I (Import key). The key copied in the previous step is pasted in; the tool then confirms the parsed agent identity — ID 001, Name win10, IP 192.168.179.142— and the key is stored after confirming with y.



```
*****
* Wazuh v4.14.5 Agent manager. *
* The following options are available: *
*****
(I) Import key from the server (I).
(Q) Quit.
Choose your action: I or Q: I

* Provide the Key generated by the server.
* The best approach is to cut and paste it.
*** OBS: Do not include spaces or new lines.

Paste it here (or '\q' to quit):
.

Agent Information:
ID: 001
Name: win10
IP Address: 192.168.179.142

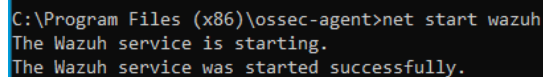
Confirm adding it?(y/n): y
Added.
```

Figure 10: `manage_agents.exe` on Windows 10: key imported and confirmed — ID 001, Name win10, IP 192.168.179.142, “Added.”

4.7 Starting the Agent Service

With the key imported, the agent service is started from the same Administrator CMD window:

```
net start wazuh
```



```
C:\Program Files (x86)\ossec-agent>net start wazuh
The Wazuh service is starting.
The Wazuh service was started successfully.
```

Figure 11: `net start wazuh` on Windows 10 returning “The Wazuh service was started successfully.”

5. Linux (Kali) Agent Deployment

Objective: Install and register a second, independent Kali Linux VM as a Wazuh agent (klin), monitored by the same manager.

Agent Package: wazuh-agent 4.14.6 (APT, official 4.x repository)

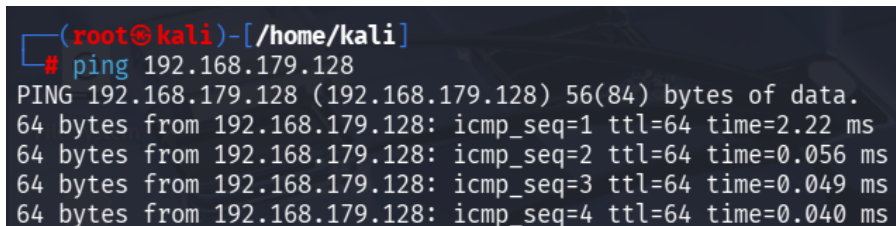
Manager IP in ossec.conf: 192.168.179.128 **Port:** 1514 **Protocol:** TCP

Config file path: /var/ossec/etc/ossec.conf

5.1 Verifying Network Connectivity

From the Linux agent VM, connectivity to the manager is confirmed with a simple ICMP test:

```
ping 192.168.179.128
```



```
(root@kali)-[/home/kali]
# ping 192.168.179.128
PING 192.168.179.128 (192.168.179.128) 56(84) bytes of data.
64 bytes from 192.168.179.128: icmp_seq=1 ttl=64 time=2.22 ms
64 bytes from 192.168.179.128: icmp_seq=2 ttl=64 time=0.056 ms
64 bytes from 192.168.179.128: icmp_seq=3 ttl=64 time=0.049 ms
64 bytes from 192.168.179.128: icmp_seq=4 ttl=64 time=0.040 ms
```

Figure 12: Successful ICMP replies from the manager (192.168.179.128) confirm the Linux agent VM and manager share the same reachable subnet.

5.2 Installing Prerequisites and Adding the Wazuh Repository

Required packages are installed, the Wazuh GPG signing key is imported, and the official APT repository is added:

```
sudo apt install curl apt-transport-https lsb-release gnupg -y

curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | \
  sudo gpg --dearmor -o /usr/share/keyrings/wazuh.gpg

echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main" | \
  sudo tee /etc/apt/sources.list.d/wazuh.list

sudo apt update
```

```
(root@kali)~[/home/kali]
# apt install curl apt-transport-https lsb-release gnupg -y
apt-transport-https is already the newest version (3.2.0+kali1).
lsb-release is already the newest version (12.1-2).
lsb-release set to manually installed.
Upgrading:
  curl  dirmngr  gnupg  gnupg-l10n  gpg  gpg-agent  gpg-wks-client  gpgconf  gpgsm  libcurl4t64  libngtcp2-crypto-openssl0
Summary:
Upgrading: 11, Installing: 0, Removing: 0, Not Upgrading: 2095
Download size: 3,669 kB
Freed space: 20.5 kB
```

Figure 13: Dependency installation on the Linux agent (curl, gnupg, apt-transport-https, and related packages).

```
(root@kali)~[/home/kali]
# curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --dearmor -o /usr/share/keyrings/wazuh.gpg
File '/usr/share/keyrings/wazuh.gpg' exists. Overwrite? (y/N) y

# echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee /etc/apt/sources.list.d/wazuh.list
deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main

# apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
Hit:2 https://packages.wazuh.com/4.x/apt stable InRelease
2095 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 14: Wazuh GPG key imported, APT repository added (packages.wazuh.com/4.x/apt), and apt update refreshing package indices.

5.3 Installing the Agent Package

```
sudo apt install wazuh-agent -y
```

Figure 15 confirms package wazuh-agent_4.14.6-1_amd64.deb (13.2 MB) is fetched, unpacked, and configured.

```
(root@kali)~[/home/kali]
# apt install wazuh-agent -y
Installing:
  wazuh-agent

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2094
Download size: 13.2 MB
Space needed: 48.4 MB / 63.1 GB available

Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.14.6-1 [13.2 MB]
Fetched 13.2 MB in 13s (1,050 kB/s)
Preconfiguring packages ...
Selecting previously unselected package wazuh-agent.
(Reading database ... 422312 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.6-1_amd64.deb ...
Unpacking wazuh-agent (4.14.6-1) ...
Setting up wazuh-agent (4.14.6-1) ...
```

Figure 15: apt install wazuh-agent -y installing version 4.14.6-1 on the Linux agent.

5.4 Pointing the Agent at the Manager

The configuration file is opened and the <server> block updated:

```
sudo nano /var/ossec/etc/ossec.conf
```



```
<server>
  <address>192.168.179.128</address>
  <port>1514</port>
  <protocol>tcp</protocol>
</server>
```

```
<ossec_config>
  <client>
    <server>
      <address>192.168.179.128</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>kali, kali2025, kali2025.4</config-profile>
    <notify_time>20</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
  </client>
```

Figure 16: `ossec.conf` on the Linux agent: manager address 192.168.179.128, port 1514, protocol TCP; config-profile `kali`, `kali2025`, `kali2025.4`.

5.5 Registering the Linux Agent on the Manager

On the manager:

```
sudo /var/ossec/bin/manage_agents
```

Option **A** is chosen; the agent is named `klin` with IP 192.168.179.143. After confirming, the tool reports **Agent added with ID 002**.

```
*****
* Wazuh v4.7.5 Agent manager.                *
* The following options are available:        *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: A

- Adding a new agent (use '\q' to return to the main menu).
  Please provide the following:
    * A name for the new agent: klin
    * The IP Address of the new agent: 192.168.179.143
  Confirm adding it?(y/n): y
  Agent added with ID 002.
```

Figure 17: `manage_agents` on the manager: agent `klin` added with IP 192.168.179.143, assigned ID 002.

5.6 Extracting and Importing the Key

Option **E** is used to list and extract the key for agent 002 (`klin`, 192.168.179.143); the available-agents list now shows both `win10` (001) and `klin` (002).


```

*****
* Wazuh v4.7.5 Agent manager. *
* The following options are available: *
*****
(A)dd an agent (A).
(E)xtract key for an agent (E).
(L)ist already added agents (L).
(R)emove an agent (R).
(Q)uit.
Choose your action: A,E,L,R or Q: E

Available agents:
  ID: 001, Name: win10, IP: 192.168.179.142
  ID: 002, Name: klin, IP: 192.168.179.143
Provide the ID of the agent to extract the key (or '\q' to quit): 002

Agent key information for '002' is:

```

Figure 18: `manage_agents` option E: agent list shows ID 001 win10 and ID 002 klin; key for 002 extracted (redacted).

On the Linux agent, the same tool is run locally and option I used to import the key:

```
sudo /var/ossec/bin/manage_agents
```

```

*****
* Wazuh v4.14.6 Agent manager. *
* The following options are available: *
*****
(I)mport key from the server (I).
(Q)uit.
Choose your action: I or Q: I

* Provide the Key generated by the server.
* The best approach is to cut and paste it.
** OBS: Do not include spaces or new lines.

Paste it here (or '\q' to quit): [REDACTED]

Agent information:
  ID:002
  Name:klin
  IP Address:192.168.179.143

Confirm adding it?(y/n): y
Added.

```

Figure 19: `manage_agents` v4.14.6 on the Linux agent: key imported and confirmed — ID 002, Name klin, IP 192.168.179.143, “Added.”

5.7 Starting and Verifying the Agent Service

```

sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent

```

```
(root@kali)-[/home/kali]
# sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent

Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' → '/usr/lib/systemd/systemd/wazuh-agent.service'.

(root@kali)-[/home/kali]
# systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-07-01 13:43:09 EDT; 15s ago
     Invocation: 55d3a63f65ff4cbd8daf4184d24057bd
   Process: 9453 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 31 (limit: 2073)
  Memory: 66.1M (peak: 70.6M)
    CPU: 14.149s
   CGroup: /system.slice/wazuh-agent.service
           └─9475 /var/ossec/bin/wazuh-execd
             └─9494 /var/ossec/bin/wazuh-agentd
               └─9515 /var/ossec/bin/wazuh-syscheckd
                 └─9536 /var/ossec/bin/wazuh-logcollector
                   └─9561 /var/ossec/bin/wazuh-modulesd
```

Figure 20: `systemctl status wazuh-agent` on klin: active (running), with child processes `wazuh-execd`, `wazuh-agentd`, `wazuh-syscheckd`, `wazuh-logcollector`, and `wazuh-modulesd`.

6. Troubleshooting: Agent/Manager Version Mismatch

Issue: The manager was installed from the legacy 4.7 install script (Wazuh 4.7.5), while both agents were installed from the current 4.x repository (Windows agent 4.14.5, Linux agent 4.14.6). Both agents remained stuck in a connection-retry loop.

Cause: Wazuh does not allow an agent to connect to a manager running an *older* version than the agent — the manager version must be equal to or newer than every connecting agent.

Fix: Upgrade the manager stack in place to the 4.x line.

Figure 21 shows the repeating warning emitted by both agents while the manager was still on 4.7.5:

```
WARNING: Unable to connect to any server.
INFO: Closing connection to server ([192.168.179.128]:1514/tcp).
INFO: Trying to connect to server ([192.168.179.128]:1514/tcp).
WARNING: Couldn't connect to server '192.168.179.128': 'Agent version must be lower or equal to manager version'
INFO: Closing connection to server ([192.168.179.128]:1514/tcp).
INFO: Trying to connect to server ([192.168.179.128]:1514/tcp).
WARNING: Couldn't connect to server '192.168.179.128': 'Agent version must be lower or equal to manager version'
INFO: Closing connection to server ([192.168.179.128]:1514/tcp).
INFO: Trying to connect to server ([192.168.179.128]:1514/tcp).
WARNING: Couldn't connect to server '192.168.179.128': 'Agent version must be lower or equal to manager version'
```

Figure 21: Agent connection log: `WARNING: Couldn't connect to server '192.168.179.128': 'Agent version must be lower or equal to manager version'` — repeated retry/failure cycle.

The fix was to upgrade the manager-side components using the already-added 4.x repository:

```
sudo apt install --only-upgrade wazuh-manager wazuh-indexer wazuh-
  dashboard filebeat
```

After the upgrade and a service restart, the manager reported version 4.14.6, matching or exceeding both agents. Both agents then transitioned from *Never connected* to *Active*.

6.1 Verifying Agent Connectivity

On the manager:

```
sudo /var/ossec/bin/agent_control -l
```

```
(root@kali)-[/home/kali]
# sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: kali (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: win10, IP: 192.168.179.142, Active
  ID: 002, Name: klin, IP: 192.168.179.143, Active

List of agentless devices:
```

Figure 22: `agent_control -l` output: ID 000 (kali, local server, Active/Local), ID 001 (win10, 192.168.179.142, Active), ID 002 (klin, 192.168.179.143, Active). All endpoints are connected.

6.2 Confirming the Upgraded Manager via the Dashboard

The Wazuh dashboard's *Server APIs* settings page independently confirms the post-upgrade manager version.

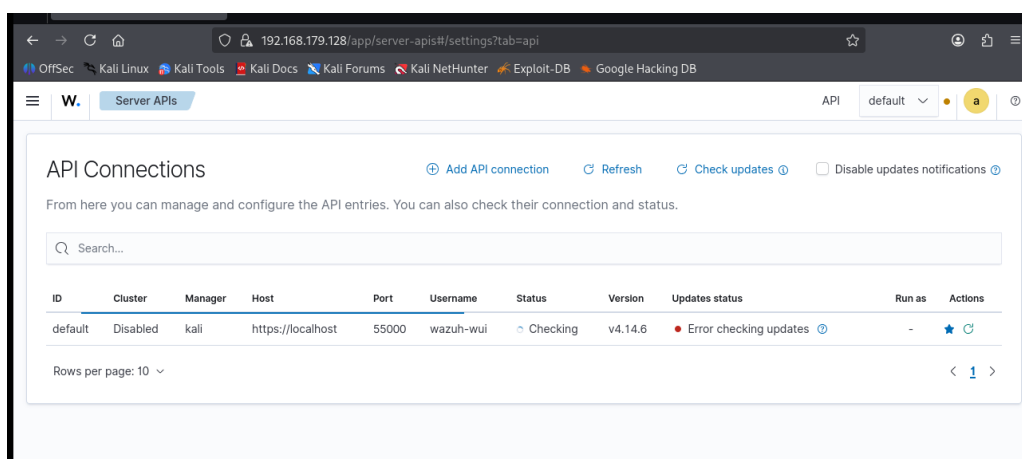


Figure 23: Dashboard **Server APIs** page (192.168.179.128/app/server-apis): default API entry pointing at manager kali, host `https://localhost:55000`, running **v4.14.6** — confirming the successful upgrade.

Symptom	Repeated “Agent version must be lower or equal to manager version”
Affected Components	win10 agent (4.14.5), klin agent (4.14.6) vs. manager (4.7.5)
Resolution Command	<code>apt install --only-upgrade wazuh-manager wazuh-indexer wazuh-dashboard filebeat</code>
Post-fix Manager Version	4.14.6
Result	Both agents changed from “Never connected” to “Active”

7. File Integrity Monitoring (FIM) on the Linux Agent

The Wazuh agent ships with FIM (Syscheck) built in; the main task is to specify which directories to monitor. FIM was configured and validated on the Linux agent, `klin` (192.168.179.143).

7.1 Enabling and Configuring Syscheck

```
sudo nano /var/ossec/etc/ossec.conf
```

Within the `<syscheck>` block, real-time monitoring of `/home` is enabled, alongside a 60-second scan frequency, scan-on-start, and alerting on newly created files:

```
<syscheck>
  <disabled>no</disabled>
  <frequency>60</frequency>
  <scan_on_start>yes</scan_on_start>
  <alert_new_files>yes</alert_new_files>
  <auto_ignore frequency="10" timeframe="3600">no</auto_ignore>
  <directories realtime="yes">/home</directories>
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>
</syscheck>
```

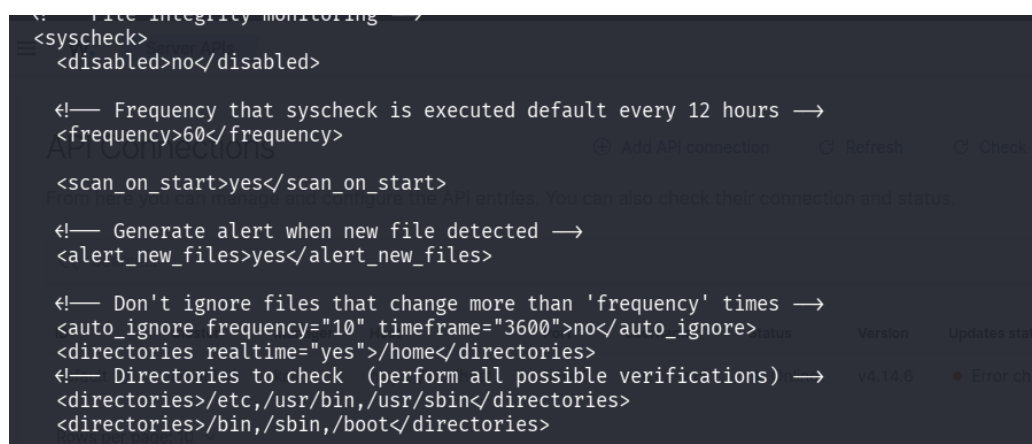


Figure 24: `<syscheck>` block in `ossec.conf` on the Linux agent: real-time monitoring enabled for `/home`, 60-second frequency, scan-on-start, and alerting on new files.

7.2 Restarting the Agent and Generating Test Activity

```
sudo systemctl restart wazuh-agent
cd ~
echo "Hello Wazuh" > test.txt
```

This creates a new file inside the monitored `/home` directory, which Syscheck's real-time watcher (via `inotify`) picks up almost immediately.

```
(root@kali)-[/home/kali]
# sudo systemctl restart wazuh-agent

(root@kali)-[/home/kali]
# echo "Hello Wazuh" > test.txt

(root@kali)-[/home/kali]
#
```

Figure 25: Linux agent restarted and a test file created with `echo "Hello Wazuh" > test.txt` inside the monitored home directory.

Subsequent modification and deletion of the same file (`echo "Modified" >> test.txt`, then `rm test.txt`) produce corresponding *File modified* and *File deleted* events, following the same detection path.

7.3 Verifying FIM Alerts in the Dashboard

In the Wazuh dashboard, under **Modules** → **File Integrity Monitoring** (or Security Events filtered on `rule.groups: syscheck`), the alerts generated by the test activity are visible.



Figure 26: File Integrity Monitoring dashboard filtered to `manager.name: kali` and `rule.groups: syscheck`. The “Alerts by action over time” panel shows a sharp spike of *added* and *modified* events at the moment the test file was created and edited; the “Top 5 agents” donut confirms `klin` as the reporting agent alongside the manager itself.

Monitored Path (real-time)	/home
Scan Frequency	60 seconds
Reporting Agent	klin (ID 002, 192.168.179.143)
Alert Types Observed	File added, File modified (added/modified visible in the action-over-time chart)
Search Filters Used	<code>rule.groups: syscheck</code> , <code>manager.name: kali</code>

8. Live Security Events — Windows Agent

Objective: Observe real-time security events from the Windows agent on the Wazuh dashboard.

Dashboard View: Threat Hunting — agent `win10` (agentId: 001), manager `kali`

Observed Total: 21 events in the selected time window

To populate the dashboard with meaningful data, routine activity was generated on the Windows 10 agent (file operations and standard command-line usage), which the agent’s log collector picks up and forwards to the manager for classification.

Figure 27 shows the Wazuh **Threat Hunting** dashboard scoped to `manager.name: kali` and `agent.id: 001` (the `win10` agent). The summary bar reports **21 total events**, **0** Level-12-or-above alerts, **0** authentication failures, and **0** authentication successes — consistent with routine system activity rather than security incidents. The “Top 10 Alert groups evolution” panel shows recurring peaks across the `windows`, `windows_application`, `windows_system`, and `policy_changed` rule groups over the observation window. The “Top 5 rule groups” donut mirrors the same four categories, and the “Top 5 PCI DSS Requirements” panel maps the activity to control **10.6**, demonstrating that Wazuh classifies routine Windows events against compliance frameworks automatically, without any custom rule authoring.

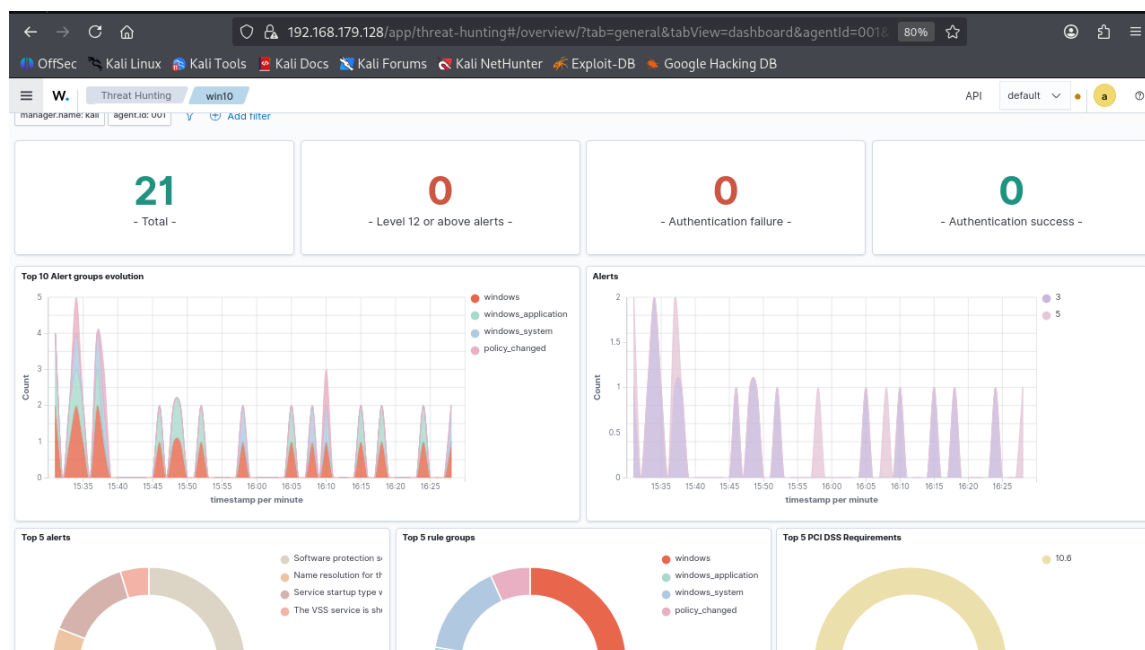


Figure 27: Wazuh Threat Hunting dashboard scoped to the `win10` agent (ID 001): 21 total events, 0 high-severity alerts, alert-group evolution across `windows` / `windows_application` / `windows_system` / `policy_changed`, and PCI DSS requirement 10.6 mapped automatically.

Metric	win10 Agent View
Total Events (window)	21
Level 12+ Alerts	0
Authentication Failures / Successes	0 / 0
Rule Groups Observed	windows, windows_application, windows.system, policy_changed
PCI DSS Control Mapped	10.6

9. Reflection & Conclusion

9.1 Summary of Results

This lab extended a single-agent Wazuh deployment into a small, mixed-OS monitoring environment. Starting from three VMware virtual machines on a shared NAT subnet, the work covered: manager installation on Kali Linux via the all-in-one script; independent agent deployment on a Windows 10 endpoint (MSI) and a second Kali Linux endpoint (APT); the five-step key-registration workflow repeated for each agent; diagnosis and resolution of an agent/manager version-mismatch fault by upgrading the manager stack from 4.7.5 to 4.14.6; File Integrity Monitoring configuration and validation on the Linux agent; and final verification of live security events from the Windows agent through the web dashboard.

The `agent_control -l` output confirmed the key milestone: both `win10` (ID 001) and `klin` (ID 002) reporting **Active** alongside the manager's own local entry (ID 000). From that point, both endpoints' events flowed into the indexer and became visible in the dashboard within seconds — FIM alerts from the Linux agent's monitored `/home` directory, and general Windows activity from the `win10` agent, automatically classified against rule groups and PCI DSS control 10.6 without any custom rule authoring.

9.2 Key Technical Insights

- **Key exchange as trust anchor:** The `manage_agents` key exchange is what makes the Wazuh architecture trustworthy — each agent has a unique cryptographic identity, and the manager will not accept events from a host that cannot present the matching key.
- **Version compatibility is enforced, not advisory:** Wazuh actively refuses connections from agents newer than the manager. Mixing an install-script-provisioned manager with repository-installed agents is a realistic and easy-to-hit failure mode that required an explicit manager upgrade to resolve.
- **Heterogeneous endpoints, one pipeline:** The same manager transparently handled a Windows MSI agent and a Linux APT agent side by side, each applying its own OS-specific `config-profile` while feeding the same indexer and dashboard.
- **FIM latency is low:** Real-time Syscheck monitoring (`realtime="yes"`) surfaced file-creation events on the Linux agent within roughly a minute of the change, which is adequate for detecting unauthorised file tampering in near real time.
- **Compliance mapping out of the box:** Wazuh automatically mapped routine Windows activity to PCI DSS control 10.6 with zero custom rules, illustrating the value of a mature default ruleset for day-one compliance reporting.

9.3 Ethical Considerations

All work in this lab was performed on privately owned virtual machines in an isolated VMware NAT environment, with no connection to the public internet during sensitive phases of the exercise. Deploying monitoring agents on any system requires explicit authorisation from the system owner — installing a Wazuh agent on a machine without consent violates the Computer Fraud and Abuse Act (US), the Computer Misuse Act (UK), and equivalent legislation including Pakistan's Prevention of Electronic Crimes Act (PECA) 2016. In a professional context, agent deployment must always be authorised, documented, and scoped to agreed endpoints only. The skills developed in this lab are intended for defensive security roles — building and maintaining monitoring infrastructure to detect and respond to threats, not to surveil systems without authorisation.